

**MINISTRY OF EDUCATION AND TRAINING  
CAN THO UNIVERSITY  
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY**



**PROJECT – FUNDAMENTAL TOPICS  
BACHELOR OF ENGINEERING IN  
INFORMATION TECHNOLOGY  
(HIGH-QUALITY PROGRAM)**

**SEPARATE SINGING VOICE FROM THE MUSIC  
SOURCE WITH DEEP LEARNING MODEL**

**Student: Bùi Dương Quốc Khánh  
Student ID: B2306669  
Class: 2023-2024 (K49)  
Advisor: Ph.D Lâm Nhật Khang**

**Can Tho, 5/2026**

**MINISTRY OF EDUCATION AND TRAINING  
CAN THO UNIVERSITY  
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY  
DEPARTMENT OF INFORMATION TECHNOLOGY**



**PROJECT – FUNDAMENTAL TOPICS  
BACHELOR OF ENGINEERING IN  
INFORMATION TECHNOLOGY  
(HIGH-QUALITY PROGRAM)**

**SEPARATE SINGING VOICE FROM THE MUSIC  
SOURCE WITH DEEP LEARNING MODEL**

**Student: Bùi Dương Quốc Khánh  
Student ID: B2306669  
Class: 2023-2024 (Cohort K49)  
Advisor: Ph.D Lâm Nhật Khang**

**Can Tho, 5/2026**

## **ACKNOWLEDGMENTS**

First of all, I want to give a gratefulness to every lecturer from College Of Information And Communication Technology for the conscientious in teaching work, so I can gain the basic knowledges and valuable experience to finish the project – fundamental topic for the high quality program of Information Technology.

I want to give a special thank to Ms. Lam Nhut Khang, who had enthusiastically support me throughout the process of doing the thesis. Your experienced knowledge and valuable advices had promoted my motivation of studying and scientific research at the university.

In spite of trying my best to finish my topic as good as possible, the thesis topic will be inevitable from the mistakes. I hope to get the evaluation and feedback from you for the improvement in the future.

I want to give you my sincere thanks for al your help.

*Can Tho, May 31<sup>st</sup>, 2026*

**Student**

**Bùi Dương Quốc Khánh**

## **ABSTRACT**

The project - fundamental topic is about applying and fine-tuning the deep learning model for separating singing voice from the music source. The work includes exploring the scientific research about the deep learning models for Music Source Separation (MSS) task. Then, I will apply and fine-tune these models to improve the accuracy and the quality of audio output. After that, the fine-tuned model will be trained with the public dataset and evaluate throughout some separating ability metrics and human evaluation. The output from the model will be two separate audio source, including a vocal source and an instrumental one. Throughout this thesis work, I hope that the result of the topic wil help improving the technology for the MSS task and motivate the researching spirit at the university.

## TABLE OF CONTENT

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>ACKNOWLEDGMENTS .....</b>                       | <b>3</b>  |
| <b>ABSTRACT .....</b>                              | <b>4</b>  |
| <b>LIST OF TABLES .....</b>                        | <b>8</b>  |
| <b>LIST OF FIGURES .....</b>                       | <b>9</b>  |
| <b>LIST OF ABBREVIATIONS .....</b>                 | <b>10</b> |
| <b>CHAPTER 1. INTRODUCTION .....</b>               | <b>11</b> |
| 1.1. THE PROBLEM OF THE STUDY .....                | 11        |
| 1.2. RELATED WORK – CITING THE PREVIOUS WORK ..... | 11        |
| 1.3. RATIONALE FOR TOPIC SELECTION.....            | 14        |
| 1.4. THE PURPOSE OF THE STUDY .....                | 15        |
| 1.5. SUBJECT AND SCOPE OF THE STUDY .....          | 15        |
| 1.5.1. Subject of the study .....                  | 15        |
| 1.5.2. Scope of the study.....                     | 15        |
| 1.6. THE STUDY METHOD.....                         | 15        |
| 1.7. OUTLINE OF THESIS.....                        | 16        |
| <b>CHAPTER 2. THEORY BASIS.....</b>                | <b>17</b> |
| 2.1. DEEP LEARNING .....                           | 17        |
| 2.2. DEEP LEARNING MODELS.....                     | 17        |
| 2.2.1. Convolutional Neural Network (CNN) .....    | 17        |
| 2.2.2. Recurrent Neural Network (RNN) .....        | 18        |

|                   |                                           |           |
|-------------------|-------------------------------------------|-----------|
| 2.2.3.            | Long Short-Term Memory (LSTM) .....       | 19        |
| 2.3.              | TRANSFORMER .....                         | 20        |
| 2.4.              | SHORT TIME FOURIER TRANSFORM (STFT) ..... | 22        |
| 2.5.              | SPECTROGRAM .....                         | 22        |
| 2.6.              | EVALUATION METRICS .....                  | 23        |
| 2.6.1.            | Source-to-distortion ratio (SDR) .....    | 23        |
| 2.6.2.            | Scale-Invariant SDR (SI-SDR).....         | 23        |
| 2.6.3.            | L1Freq.....                               | 23        |
| 2.6.4.            | Fullness & Bleedless .....                | 24        |
| 2.7.              | GOOGLE COLAB.....                         | 24        |
| 2.8.              | KAGGLE .....                              | 24        |
| <b>CHAPTER 3.</b> | <b>IMPLEMENTATION METHOD .....</b>        | <b>25</b> |
| 3.1.              | OVERALL MODEL .....                       | 25        |
| 3.2.              | DATASET.....                              | 28        |
| 3.3.              | PREPROCESSING DATASET .....               | 29        |
| 3.4.              | TRAINING PROCESS .....                    | 29        |
| 3.4.1.            | Device Information.....                   | 29        |
| 3.4.2.            | Dataset Splitting .....                   | 29        |
| 3.4.3.            | Training Parameters.....                  | 30        |
| 3.4.4.            | Deployed Steps.....                       | 31        |
| 3.5.              | EVALUATE MODEL .....                      | 33        |

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>CHAPTER 4. EXPERIMENT AND EVALUATE .....</b> | <b>34</b> |
| 4.1. MODEL RESULT .....                         | 34        |
| 4.2. EVALUATION .....                           | 38        |
| 4.3. DISCUSSION .....                           | 39        |
| <b>CHAPTER 5. CONCLUSION .....</b>              | <b>41</b> |
| 5.1. OVERALL .....                              | 41        |
| 5.2. DEVELOPMENT DIRECTION.....                 | 41        |
| <b>REFERENCES.....</b>                          | <b>42</b> |

## **LIST OF TABLES**

|          |                                                                    |    |
|----------|--------------------------------------------------------------------|----|
| Table 1. | The structure of Mel-Band RoFormer Model in this project .....     | 26 |
| Table 2. | The table of model results comparison in different epochs.....     | 34 |
| Table 3. | The validated results of some Music Source Separation models ..... | 38 |

## LIST OF FIGURES

|            |                                                                              |    |
|------------|------------------------------------------------------------------------------|----|
| Figure 1.  | A convolutional layer in Convolutional Neural Networks.....                  | 17 |
| Figure 2.  | An example of pooling layer in Convolutional Neural Networks.....            | 18 |
| Figure 3.  | A structure of Recurrent Neural Network .....                                | 18 |
| Figure 4.  | The architecture of Transformer model.....                                   | 21 |
| Figure 5.  | The diagram of Mel-Band RoFormer.....                                        | 25 |
| Figure 6.  | The list of song folders in the MUSDB18-HQ's train dataset .....             | 28 |
| Figure 7.  | The list of song folders in the MUSDB18-HQ's test dataset .....              | 29 |
| Figure 8.  | The list of song folders used for training process.....                      | 30 |
| Figure 9.  | The list of song folders used for testing process .....                      | 30 |
| Figure 10. | The diagram of training loss until the 45 <sup>th</sup> epoch.....           | 35 |
| Figure 11. | The diagram of SDR metric values until the 45 <sup>th</sup> epoch .....      | 36 |
| Figure 12. | The diagram of L1Freq metric values until the 45 <sup>th</sup> epoch .....   | 36 |
| Figure 13. | The diagram of Bleedless metric values until the 45 <sup>th</sup> epoch..... | 37 |
| Figure 14. | The diagram of Fullness metric values until the 45 <sup>th</sup> epoch.....  | 37 |

## LIST OF ABBREVIATIONS

| N.O | Abbreviation | Meaning                             |
|-----|--------------|-------------------------------------|
| 1   | AI           | Artificial Intelligence             |
| 2   | MSS          | Music Source Separation             |
| 3   | MIR          | Music Information Retrieval         |
| 4   | CNN          | Convolutional Neural Networks       |
| 5   | ReLU         | Rectified Linear Unit               |
| 6   | BSRNN        | Band-Split Recurrent Neural Network |
| 7   | RNN          | Recurrent Neural Network            |
| 8   | MLP          | Muti-Layer Perceptron               |
| 9   | LSTM         | Long-Short Term Memory              |
| 10  | NLP          | Natural Language Processing         |
| 11  | SE           | Spectral Embedding                  |
| 12  | TE           | Temporal Embedding                  |
| 13  | RoPE         | Rotary Position Embedding           |
| 14  | SDR          | Signal-To-Distortion Ratio          |
| 15  | STFT         | Short Time Fourier Transform        |
| 16  | TP           | True Positive                       |
| 17  | FP           | False Positive                      |
| 18  | FN           | False Negative                      |
| 19  | GPU          | Graphic Processing Unit             |
| 20  | TPU          | Tensor Processing Unit              |
| 21  | SDR          | Source-to-distortion ratio          |
| 22  | SAR          | Source-to-artifacts ratio           |
| 23  | SIR          | Source-to-interferences ratio       |
| 24  | GLU          | Gated Linear Unit                   |
| 25  | MSST         | Music-Source-Separation-Training    |
| 26  | Hz           | Heinz                               |
| 27  | VRAM         | Video RAM                           |

## **CHAPTER 1. INTRODUCTION**

### **1.1. THE PROBLEM OF THE STUDY**

Music have a long history of development. It is originated from the beginning of the human revolution, when music was generated from the daily materials such as bamboo, animal bones and skin [1]. Throughout the thousand years, music has been threaded in the cultural life, religion and belief of various races all around the world. It isn't only a form of entertaining, it is also a way to represent the personality, cultural character and local traditional value. In recent years, due to the rapid development of social network, music has become an indispensable in a human life, and risen up to be a huge industry in the world.

In the same period, the development of global technology has contributed in various field of the human life in, including music. Since 1980s, early digital recording system had become a watershed moment for music technology development in the future [2]. Throughout many decades, these technologies has been accessed to a wide range of people. The quality of sound source has continuously upgrade, with the combination of harmony, instrumental and recording method to diverse our musical experience. Nowadays, with the development of AI domain, music has been acknowledge to have significant impacts on generating task despite of arguments about the copyright and artistic creative stamp.

Many researchs about musical task in AI domain has been populared since the beginning of 2010s, with a lot of potential to develop. And MSS task is one of the most concerned research fields, where a file or a source of mixture sound is separated to extract different musical components such as vocal, drum, bass, guitar and other kind of instruments. By exploring and applying deep learning model, MSS task had contributed to many music-related work such as generating automatic karaoke version, remixing. However, there still have some challenges in separating music source such as background noise, the overlap between vocal and instrumental frequency, the complication in the audio signal. Therefore, developing and upgrading the deep learning models in MSS task is necessary to improve the quality of sound source and benefit our music-related task.

### **1.2. RELATED WORK – CITING THE PREVIOUS WORK**

The development of deep learning models had premised for solving MSS task. In the last decade, many research papers about separating vocal from the music source problem have been proposed.

Chandna and co-workers [3] introduced a Convolutional Neural Network (CNN) for monoaural audio source separation. The model includes two stages: an encoding stage consists of two convolutional layers and a fully connected dense layer acting as a bottleneck to compress information, a decoding stage with another

fully connected layer, a ReLU non-linearity and the same size as the output of the second convolutional layer. After that, this layer is reshaped to the same dimensions as the horizontal convolutional layer and pass through successive deconvolution layers. This work is repeated to compute estimates for each of the sources. The model was trained and tested on DSD100 datasets and witnessed a good performance.

In the same year, Jansson's research team [4] had developed model based on the fully convolutional network and is similar to the deconvolutional network, where each convolutional layers in a stack halves the size of the image but doubles the number of channels to encode the image into a small and deep representation. Then, that encoding is decoded to the original size of the image by a stack of upsampling layers. Besides, the research team had adapted the U-Net architecture, initially developed biomedical imaging task, to add an additive skip connection between the same hierarchical level in the encoder and decoder of the model. This allows low-level information to flow directly from the high-resolution input to the high-resolution output. The model was trained on the dataset of approximately 20,000 pairs of original – instrumental sounds, and evaluated on iKala and MedleyDB datasets, compared the performance with two other CNN models. The result shows that U-Net architecture model outperformed two other models. Besides, the research team also did a survey on an online platform, where a respondent will hear three different audio clips generated from three mentioned models in a random order. After that, a respondent will judge these audio clips based on the reference clip according to seven steps from "Poor" to "Perfect". The surveyed result from 99 respondents and about 1,500 responses shown the outstanding performance in the U-Net architecture model.

In 2022, Yi Lou and Jianwei Wu [5] proposed Band-Split RNN (BSRNN) model, a frequency-domain model designed for high sample rate signals with flexible and explicit segregation and modeling of different frequency bands. BSRNN splits the spectrogram into a set of subbands with a set of determined bandwidths. These bandwidths are adjusted for different instrumental sounds. The subband spectrogram are transformed to generate a series of features with same dimension, and stacked residual Recurrent Neural Network (RNN) are utilized to perform interleaved cross-band and cross-sequence modeling similar to dual-path networks for speech separation. Each subband feature in the output of the last residual RN layer is transformed by a multi-layer perceptron (MLP) to generate its corresponding complex-valued time-frequency mask to generate the estimated target source spectrogram. All that spectrograms are concatenated to form the full spectrogram of the target source. The model includes band split module, band and sequence modeling module and mask estimation module. Band split module is the most important module quan, allows us to incorporate prior knowledge on the target source into the model design. BSRNN model was trained, fine-tuned and evaluated on MUSDB18 and MUSDB18-HQ dataset with uSDR và cSDR evaluated metrics. The result shows that the models had the outstanding performance compared to

other MSS systems. Besides, the semi-supervised finetuning pipeline can improve the representation of songs in different types.

The research team of Défossez [6] had experimented two architectures for MSS tasks in the waveform domain: Demucs and Conv-Tasnet. The Conv-Tasnet architecture, initially developed for separating voice in short speech, was fine-tuned by increasing the size of kernel and the number of channels in encoder and decoder. It was seen an outstanding performance on short duration audio sources, but not seen similarly result on the entire song because of global normalization. The proposed Demucs architecture with BiDirectional LSTM 2 in the encoder and the convolutional decoder connected with U-Net architecture. The experiment on MusDB dataset shows that by fully enhanced data, Demucs had better results than other architectures with the average SDR score of 6.3. Besides, based on the human evaluation, Demucs had generated the audio source with naturalness sound despite being affected with the problems between the vocal and other audio sources.

Since the proposal of Transformers architecture [7], it had proved a significant effort not only Natural Language Processing (NLP) tasks but also in Music Information Retrieval (MIR), with the outstanding performance in modeling the sequences to predict the high-level sound semantics. In 2021, Lu's research team [8] introduced SpecTNT, a Transformer model to model both spectral and temporal sequences of an input time-frequency representation. In detail, the Transformer in Transformer architectures, initially applied in visual recognition tasks, is designed to process the audio data. The SpecTNT block holds two data flows: spectral embedding (SE) and temporal embedding (TE) respectively processed with two Transformer encoders. Spectral Transformer extracts the frequency-related features into the Frequency Class Token for each frame. After that, these FCTs are linearly projected and added to the Temporal Embedding (TEs) to aggregate useful. Then, a temporal Transformer processes TEs to exchange information across the time axis. The model was trained on 1K and MedleyDB for vocal melody extraction task, and evaluated on ADC2004, MIREX05 and MedleyDB datasets. The result shows that SpecTNT gained the good result, outstanding the performances of other CNN and CRNN models.

In 2023, the research team of Lu and Wang [9] introduced a novel approach of waveform domain based on Band-Split RoPE Transformer (BS-RoFormer) model, combining the ideas of band-split and hierarchical Transformer. The Band-Split was inherited from the BSRNN [5] model by dividing the audio source into the number of uneven non-overlapping subbands along the frequency axis and applying individual MLP for each subband. Besides, the model also adopts Rotary Position Embedding (RoPE) to significantly improve the performance. The model was trained and evaluated on MUSDB18-HQ dataset. The output of the separated audio source was evaluated on the signal-to-distortion (SDR) metric to judge the separation ability, shown that the model had an outstanding performance. This research had shed the light on the development of the future MSS systems.

In 2024, Wang and co-workers [10] introduced Mel-RoFormer, a spectrogram-based model featuring two key designs: a Mel-band Projection module at the front-end to enhance the model's capability to capture informative features across multiple frequency bands, and interleaved RoPE Transformers to explicitly model the frequency and time dimensions as two separate sequences. The model is a successor of BS-RoFormer [9] model with an enhanced band-division scheme that leverages Mel-scale to improve the model's generalization ability. The model was experimented on the public datasets for vocal separation and vocal melody transcription tasks. The result shows that the model had promising results in both tasks, proving the efficiency in modeling the complicated audio signal.

Until 2026, Solovyev and co-workers [23] had introduced Music-Source-Separation-Training (MSST), an open-source universal framework for MSS tasks. Starting development since 2023 on GitHub platform [24], MSST is seemed as a framework that unifies training, validation and inference process for a wide range of modern model architecture within a single, configuration-driven interface. For detail, this framework supports building multiple architectural patterns from convolutional networks to Transformers, parameters for dataset preprocessing and augmentations, loss functions, evaluation metrics and fine-tuned techniques that improve the ability and quality of music source separation tasks. By unifying these components into a reproduced framework, MSST has simplified the experimental and deployment of the researching community for rapidly making an idea into realistic results. Recently, this framework has still persistently developed and upgraded to diversify and optimize our MSS tasks researching process. Simultaneously, the versatile and convenience of framework has attracted a huge number of global researchers by providing more optimized experimented results and widening the researching community. From these above advantages, MSST is applied as the main inspiration for the whole project.

### **1.3. RATIONALE FOR TOPIC SELECTION**

Nowadays, with the rapid development of social network, music has become an essential part of the human life. Following that flow, many form of music-related entertaining such as karaoke, remixing the released music product is being popularized for different range of ages. To maintain that development flow, it is necessary to continuously create more musical product for the entertaining purpose.

Processing the musical audio sources is a concerned problem to ensure the good quality of the sound and fitted with the experience of human when they are listening to music or entertaining. In more specific, there are many concern about how to separate singing voice from a source of musical sound mixture for various purposes. However, many technologies for MSS task isn't widely popular with the masses and still have many problems. For example, an open-source audio editor and recording tool called Audacity requires some manual act such as filtering and reduce

the frequency to remove the background sound and retain the voice. These works are expected some professional knowledge and experimental to generate the high quality and accurate sound. Moreover, the complication in the frequency of the audio source can significantly affect to the performance and accuracy in the manual process. With the risen of AI field, many deep learning model had been developed and significantly solved the MSS problem. However, there still have various barrier related to the quality of the audio source output such as the overlap between the frequency of vocal and instrumental, the complicated of audio signal. Therefore, it is necessary to develop and upgrade the deep learning models to enhanced the ability of separating musical sources and the quality of audio sample output, bringing more effectiveness to many musical-related works.

#### **1.4. THE PURPOSE OF THE STUDY**

The target of the study is to explore and research the method, applying deep learning model for music source separation (MSS) task. The expected output of the model will be two separate audio source, including one vocal source and one instrumental source. Throughout this topic, the audio processing task will be improved and the result of the topic will be developed and widely applied in the future. Besides, the topic will also motivate the scientific researching spirit at the university.

#### **1.5. SUBJECT AND SCOPE OF THE STUDY**

##### **1.5.1. Subject of the study**

The subject of the study is the deep learning models for separating singing voice from the music sources.

##### **1.5.2. Scope of the study**

The scope of the study is to applying and fine-tuning the deep learning models for music source separation (MSS) task. The topic is focus on separating the singing voice from the music sources to generate two separate audio source, including a vocal audio source and an instrumental source.

#### **1.6. THE STUDY METHOD**

Firstly, I will explore some scientific research paper about the music information retrieval (MIR) and music source separation (MSS) tasks. To be more specific, the content of these research paper will be about some technologies and proposed deep learning model for solving the topic problem. After that, I will choose one suitable model and experiment it by fine-tuning and upgrade it to improve the accuracy and performance of the model. By the way, some datasets will be prepared for training and testing the model, which are the public datasets and self-collected

data from the social network platform. In the next step, the output of the model will be evaluated throughout the human evaluation and some well-known metrics for sound separating ability. And finally, I will have more detailed comment about the topic result and propose some potential development direction for the future,

## **1.7. OUTLINE OF THESIS**

**Chapter 1:** Introduction

**Chapter 2:** Theory basis

**Chapter 3:** Implementation Method

**Chapter 4:** Experimental and Evaluate

**Chapter 5:** Conclusion

## CHAPTER 2. THEORY BASIS

### 2.1. DEEP LEARNING

Deep Learning [11] is a subset field of machine learning, applying to help machines understand, learn and interact with complex. In detail, the core architecture of deep learning is ANN (Artificial Neural Networks), which is inspired by the structure of human brain. This neural network includes many interconnected nodes or “neurons” layers. Each layer performs the mathematical operation to process data. A deep neural network will include the input layer to receive data. Then, these input data will be processed and passed through hidden layers for information processing in different levels using nonlinear functions. Therefore, these input data will be transformed into the predicted output of the model.

### 2.2. DEEP LEARNING MODELS

#### 2.2.1. Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN) [12] is a neural network architecture inspired by human vision system. Initially developed for image processing tasks, this architecture is also applied in audio tasks. The architecture of CNN usually includes three layers: convolutional layer, pooling layer and fully-connected layer.

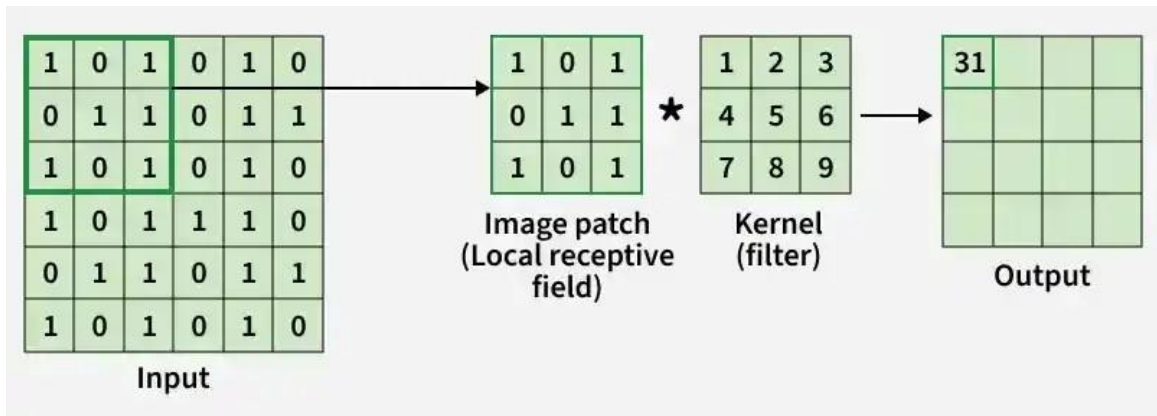


Figure 1. A convolutional layer in Convolutional Neural Networks

The convolutional layer plays a vital role in how CNN work. This layer use kernels, a small dimensional matrix, as a sliding window to glide through the input. By that way, the value in each position that kernel glides through will be calculated with the value in the corresponded position in kernel to generate a value in the corresponded position of the output matrix. The convolutional layer can significantly reduce the complexity of the model by optimizing output. Many methods can be performed for this work such as increase the stride that kernel can glide through the input or padding zero surrounded the input.

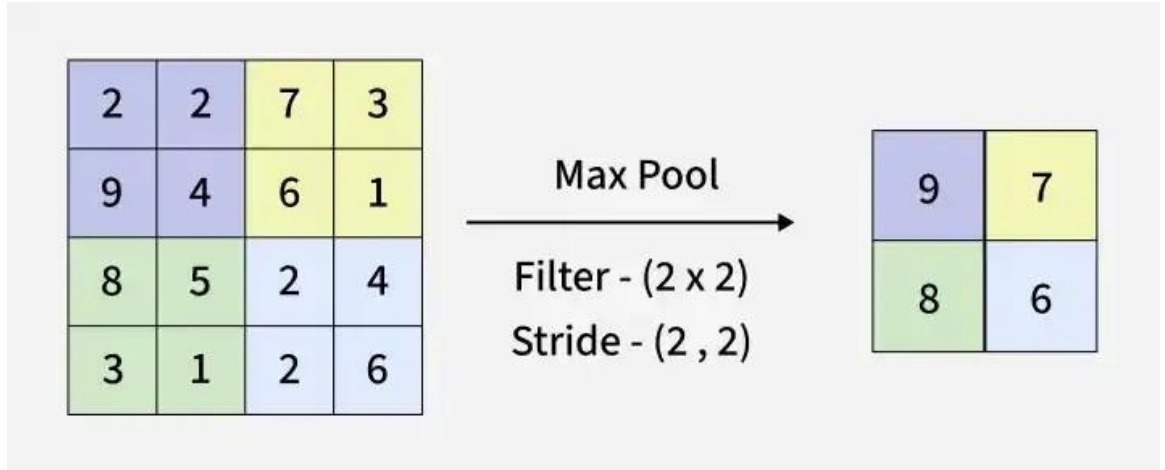


Figure 2. An example of pooling layer in Convolutional Neural Networks

The pooling layer reduces the dimensionality of the input representation, and further reduce the number of parameters and the computational complexity of the model. This layer will operate over each activation map in the input and computes the value in each region of the map. This works will help reducing the size of activation map and capturing some important features or overall contexts.

The fully-connected layer, as known as Dense layers, has a similar role to the ANN network. It includes many neurons which are directly connected to the adjacent neurons. This layer will perform high-level reasoning from extracted features and produces the output results from the activation functions for the final reasoning.

### 2.2.2. Recurrent Neural Network (RNN)

Recurrent Neural Network (RNN) [13] is a neural network which aims to detect patterns in a sequence of data.

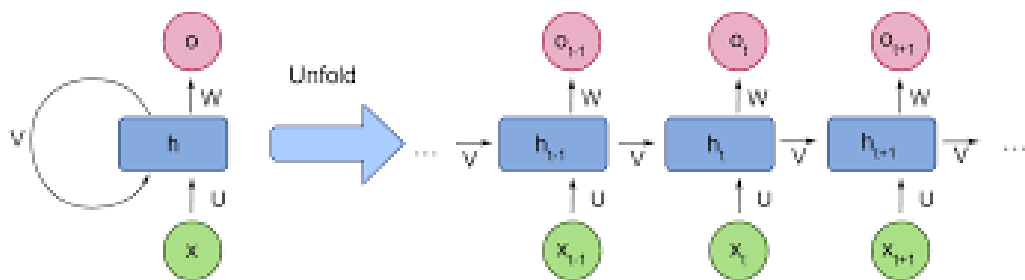


Figure 3. A structure of Recurrent Neural Network

RNN includes two main components, recurrent unit and unfolding. A recurrent unit holds a hidden state to maintain information about previous inputs in a sequence. These units can remember information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time. About unfolding, this is a process of expanding the recurrent structure over time steps. In

each time step, RNN will process the current input and a hidden state from the previous time step. This helps the model remember the previous data points and use that information to affect the current output. This unrolling enables backpropagation through time (BPTT), a learning process where errors are propagated across time steps to adjust the network's weights, to enhance the ability of the model to learn dependencies within sequential data.

However, one of the problems in RNN is that it finds it challenging to explore long-term dependencies. When we train a model through time, the learnable gradient can be minimized through many steps, making it difficult to learn long-term dependencies because of the unrelation with the previous information. And sometimes, exploding gradients can uncontrollably occur causing an unstabilized learning process of the model.

### 2.2.3. Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) [14] is an improved version of the RNN model, designed to solve the short-term memory in the model. In detail, the structure uses the memory cells and gates to control the information flow through the network.

Memory cell is a composite unit built from simpler nodes, which includes a multiplicative node working as gates to control the flow of information through the network. A LSTM cell includes three gates: forget gate, input gate and output gate.

The forget gate decides which information should be kept or removed from the cell state. It uses the current input and previous hidden state, then applies a sigmoid function to generate values between 0 and 1.

The equation for the forget gate is represented by the following function:

$$f_t = \sigma(W_f \cdot [h_{T-1}, x_t] + b_f)$$

Where:

- $W_f$  represents the weight matrix associated with the forget gate.
- $[h_{T-1}, x_t]$  denotes the concatenation of the current input and the previous hidden state.
- $b_f$  is the bias with the forget gate.
- $\sigma$  is the sigmoid activation function.

The input gate adds the useful information to a cell state. This addition will be through two layers. Firstly, the information is adjusted by using a sigmoid function and filtering the necessary remembered value corresponding to the forget gate.

$$i_t = \sigma(W_i \cdot [h_{T-1}, x_t] + b_i)$$

In this step, if the sigmoid function generates the zero value, that information won't be kept. If the sigmoid generates the 1 value, the information is important to maintain.

The next layer is responsible for the short-term memory and the input data, represented by the tanh function:

$$\hat{C}_t = \tanh(W_c \cdot [h_{T-1}, x_t] + b_c)$$

Finally, the result of two layers will be multiplied to generate the useful information. In this step, we apply element-wise multiplication for the previous state and forget gate result to filter the information that we had decided to ignore before. Then, we add the new candidate values scaled by how much we decided to update each state value.

$$C_t = f_t \odot C_{T-1} + i_t \odot \hat{C}_t$$

Output Gate determined which information from the current cell state should be passed as the hidden state at the current time step. It uses the previous hidden state and the current input, followed by a sigmoid to control the flow of output. An equation for this phase is represented as:

$$o_t = \sigma(W_o \cdot [h_{T-1}, x_t] + b_o)$$

Next, the current cell state is passed through a tanh activation function to scale its value between -1 and +1. Finally, this cell state is multiplied element-wise with the output gate activation to produce the hidden state:

$$h_t = o_t \odot \tanh(C_t)$$

The hidden state is then passed to the next time step and can also be used to generate the output of the network.

### 2.3. TRANSFORMER

Transformer [7] is a model structure based on the self-attention mechanism to process and effectively generate sequences. By this mechanism, the model can capture the long-range dependencies and contextual relationship. The structure of the model based on the encoder-decoder architecture, allowing model to parallelized process the input. This structure isn't only proved an efficient in NLP tasks, but also some MIR tasks because of its excellent performance in modeling sequences to predict high-level music semantics.

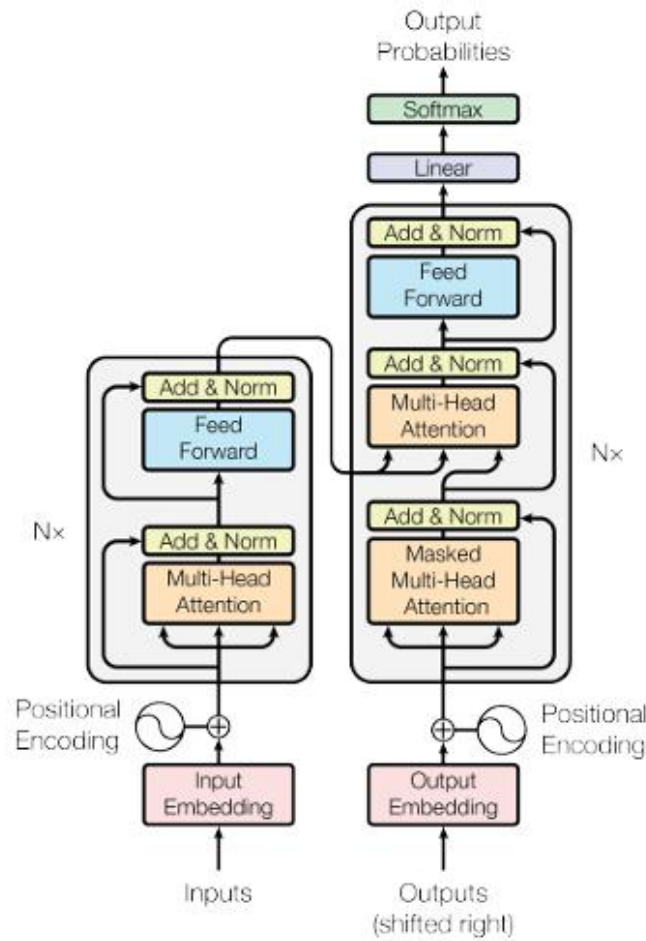


Figure 4. The architecture of Transformer model

Encoder has a role to generate a high dimensional representation of the input sequences that decoder can use to generate the output. It is composed of a stack of identical layers. Each layer has two sub-layers. The first sub-layer is a multi-head self-attention mechanism, allowing model to considered the information from multiple subspaces representation in different positions. The second one is a feed-forward network including two linear transformations with a ReLU activation in between. This sub-layer processes the output of the self-attention mechanism to generate a refined representation. These two sub-layers are surrounded by a residual connection and layer normalization, ensuring stability and improve coverage during training.

Similar to encoder, decoder also includes multiple identical layers. Its primary mission is to generate the output sequence based on the representations provided by the encoder and the previously generated tokens of the output. Besides, a masked multi-head self-attention layer is added to prevent attending to future tokens to maintain to autoregressive property.

## 2.4. SHORT TIME FOURIER TRANSFORM (STFT)

Short Time Fourier Transform (STFT) [15] is a Fourier-related transform used to determine the sinusoidal frequency and phase content of local sections of a signal as it changes over time. While the traditional Fourier transform only gives the information about the frequency domain without any information about the time, STFT can let us know when the frequency occurs in the time domain.

The calculation procedure of STFT is to divide a longer time signal into shorter segments of equal length and then compute the Fourier transform separately on each shorter segment.

In the continuous-time-case, the transformed function is multiplied by a window function which is non-zero for only a short period of time. The Fourier transform of the resulting signal is taken, then the window is slid along the time axis until the end leads to a two-dimensional representation of the signal. This case is represented by this function:

$$\text{STFT}\{x(t)\}(\tau, \omega) \equiv X(\tau, \omega) = \int_{-\infty}^{\infty} x(t)w(t - \tau)e^{-i\omega t} dt$$

Where:

- $w(t)$  is the window function, commonly a Hann window or Gaussian window
- $x(t)$  is the signal to be transformed
- $X(\tau, \omega)$  is the Fourier Transform of  $x(t)w(t - \tau)$ , a complex function representing the phase and magnitude of the signal over time and frequency

In the discrete time case, the transformed data could be divided into overlapped frames to reduce artifacts at the boundary. Each frame is applied Fourier transform, and the complex result is added to a matrix, which records magnitude and phase for each point in time and frequency. This case is represented as the equation:

$$\text{STFT}\{x[n]\}(m, \omega) \equiv X(m, \omega) = \sum_{n=-\infty}^{\infty} x[n]w[n - m]e^{-i\omega n}$$

In this case,  $m$  is discrete and  $\omega$  is continuous, but in most typical applications the STFT is performed using fast Fourier Transform, so both variables are discrete and quantized.

If it is necessary to have only a small number of  $\omega$ , or the STFT is desired to be evaluated for each shift of the window, STFT may be more efficiently evaluated by using a sliding discrete Fourier function algorithm.

STFT is a powerful calculator to transform the audio signal from the time domain to the frequency domain, which is applied in audio signal processing.

## 2.5. SPECTROGRAM

Spectrogram [16] is a visual representation of the spectrum of a signal as it varies with time. In detail, the horizontal axis represents for the time, while the vertical axis shows the frequency and color to represent the intensity of the

frequency in a specific time. The spectrogram is generated by dividing the signal in the time-domain form into multiple smaller frame and applying STFT, and then calculate the square of each frequency amplitude in each moment.

## 2.6. EVALUATION METRICS

### 2.6.1. Source-to-distortion ratio (SDR)

Source-to-distortion ratio (SDR) [18] is the measurement to compare the quality of the target audio source and the separated version of the source to measure the operative of the model.

$$SDR := 10 \log_{10} \left( \frac{\|s_{target}\|^2}{\|e_{interf} + e_{noise} + e_{artif}\|^2} \right)$$

Where  $s_{target}$  is the true audio source, while  $e_{interf}$ ,  $e_{noise}$ ,  $e_{artif}$  represent for the error types for interferences, noises and artifacts.

### 2.6.2. Scale-Invariant SDR (SI-SDR)

Scale-Invariant SDR (SI-SDR) [25] is a variant of SDR that is invariant to overall scale (loudness), eliminating trivial normalization differences. It quantifies the distortion between an estimated signal  $\hat{s}$  and a reference clean signal  $s$  after optimally rescaling the reference. SI-SDR is also suitable as a differentiable training loss.

$$\alpha = \frac{\langle \hat{s}, s \rangle}{\|s\|^2}$$

$$SI - SDR := 10 \log_{10} \left( \frac{\|\alpha s\|^2}{\|\hat{s} - \alpha s\|^2} \right)$$

Where  $\alpha s$  is a projection of  $\hat{s}$  onto  $s$ , and the numerator quantifies the energy in the “target” component, while the denominator measures the residual error.

### 2.6.3. L1Freq

L1Freq [26] is a spectral metric comparing the magnitude spectrograms of the reference and estimated audio signals using the Short-Time Fourier transform (STFT) and calculating the L1 loss between them. The result is scaled to the range of [0, 100]. In detail, STFT is defined as follow:

$$STFT(x_t) = X_{n,\omega}$$

Where  $n$  and  $\omega$  stand for frame and frequency bin indices, respectively.

Many source separation models estimate a magnitude spectrogram  $|\bar{Y}_{n,\omega,k}|$  per source  $k$ , and rely on standard regression losses like L1 or L2:

$$L1_{freq} = \frac{1}{N\Omega K} \sum_{n,\omega,k} \left| |\bar{Y}_{n,\omega,k}| - |Y_{n,\omega,k}| \right|,$$

$$L2_{\text{freq}} = \frac{1}{N\Omega K} \sum_{n,\omega,k} \left| |\bar{Y}_{n,\omega,k}| - |Y_{n,\omega,k}| \right|^2$$

#### 2.6.4. Fullness & Bleedless

Fullness and Bleedless are two metrics that use magnitude spectrogram differences between estimated and reference signals, mapped to a mel-frequency scale and decibel (dB) domain to align with human auditory perception.

The Bleedless metric quantifies how much of the estimated signal bleeds into the reference signal, while the Fullness metric evaluates how completely the target source has been preserved. Both metrics are normalized to a range of 0 and 100 with higher values indicating better performance.

### 2.7. GOOGLE COLAB

Google Colaboratory [19], or Google Colab, is a free cloud-based platform provided by Google. It allows users to write and execute Python code through the browser, especially suited for many tasks in machine learning, data analysis and education. It requires no setup or computer configuration, everything can be run through the browser.

Some notable features of Google Colab can be listed as follows:

- Colab provides free access to the computer resources, including GPU (Graphic Processing Unit) and TPU (Tensor Processing Unit). However, because it is free service, Colab will have some limitation about the usage of system resources can be used and time.
- Users can easily use and access, directly connect to the other platforms such as Google Drive, GitHub and so on.
- Colab is compatible with multiple Python libraries used in machine learning, data analysis and visualization, such as TensorFlow, PyTorch, Matplotlib and so on.
- The Notebook environment in Colab can be shared, allowing multiple users to collaborate and access through the link. By that way, many users can fix and working on Notebook at the same time.

### 2.8. KAGGLE

Kaggle [20] is an online platform owned by Google, especially applying for various tasks of data science, machine learning and AI. This platform enables users to find and upload datasets in different domains, participate in competition to solve realistic challenges in AI and data science. Similar to Google Colab, Kaggle also supports many free computational resources to enhanced the hardware performance such as GPU and TPU. Besides, it is also integrated many popular Python libraries for machine learning such as scikit-learn, TensorFlow, PyTorch, Pandas, Matplotlib and so on.

## CHAPTER 3. IMPLEMENTATION METHOD

### 3.1. OVERALL MODEL

To build the model for music source separation task, we use the Mel-Band RoFormer [10] model from the research of Wang and co-workers. The models had been experimented by the research team on the vocal separation task and achieved an excellent results compared to some contemporary models.

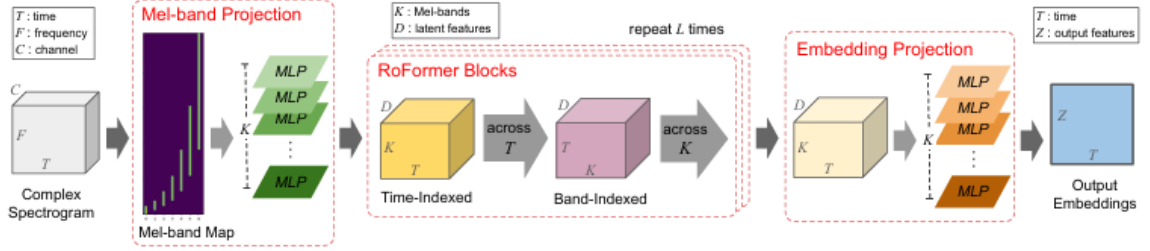


Figure 5. The diagram of Mel-Band RoFormer

The input of the model is a complex spectrogram with dimensions denoted the number of channels, frequency bins, and time steps. This frequency-time representation is obtained via STFT algorithm. This spectrogram is processed through three big modules: Mel-Band Projection, RoFormer blocks and Embedding Projection.

- **Mel-band Projection** comprises a frequency-to-Mel-band mapping and a set of multi-layer perceptron (MLP). This module is conceptualized as a learnable Mel filter-bank, with the MLP layer functioning as the mechanism to learn the filters. This grants the model greater flexibility in determining the optimal shapes for different filters without being confined to predefined filter designs.

- **RoFormer blocks**, referred as RoPE Transformer blocks, is inherited from the Band-Split Transformer [9] model. In detail, the block consists of a stack of interleaved RoPE Transformer encoders. The interleaved sequence modeling processes the data across time and sub-band dimensions. In this case, RoPE plays a vital role in enhancing the performance of Transformer by preserving the positional information within the sequence, making it invariant to repetitive processes of rearrangement.

- **Embedding Projection** module plays a crucial role in generating suitable embedding necessary for various downstream tasks. Module includes multiple MLP networks, which lead to more effective mask estimation. Each MLP contains RMSNorm layer, a linear layer followed by a Tanh activation to transform from D to 4D dimensions. Another linear layer in MLP is followed by a gated linear unit (GLU) to transforms from 4D to a desired length. This length will be used to specified the number of output features for specific purpose. The output of MLPs are concatenated along the feature dimension to generate the final result.

After go through three modules, the output of the model is the embedding with the dimensions of output features and time steps.

The structure of the model is shown on Table 1:

| Layer (type)                             | Output Shape         | Param #     |
|------------------------------------------|----------------------|-------------|
| MelBandRoFormer                          | [1, 2, 131418]       | --          |
| RMSNorm-1                                | [1, 299, 28]         | 0           |
| Linear-2                                 | [1, 299, 384]        | 11, 136     |
| ...                                      |                      |             |
| BandSplit-121                            | [1, 299, 60, 384]    | 0           |
| RMSNorm-122                              | [1, 299, 384]        | 0           |
| Linear-123                               | [1, 299, 1536]       | 589, 824    |
| Rearrange-124                            | [1, 120, 8, 64, 299] | 0           |
| Attend-125                               | [1, 8, 64, 299]      | 0           |
| Rearrange-126                            | [1, 299, 512]        | 0           |
| Linear-127                               | [1, 299, 384]        | 196, 608    |
| LinearAttention-128                      | [1, 299, 384]        | 0           |
| RMSNorm-129                              | [1, 299, 384]        | 0           |
| Linear-130                               | [1, 299, 1536]       | 591, 360    |
| GELU-131                                 | [1, 299, 1536]       | 0           |
| Dropout-132                              | [1, 299, 1536]       | 0           |
| Linear-133                               | [1, 299, 384]        | 599, 208    |
| Dropout-134                              | [1, 299, 384]        | 0           |
| FeedForward-135                          | [1, 299, 384]        | 0           |
| RMSNorm-136                              | [1, 299, 384]        | 0           |
| Transformer-137                          | [1, 299, 384]        | 0           |
| RMSNorm-138                              | [1, 60, 384]         | 0           |
| Linear-139                               | [1, 60, 1536]        | 589, 824    |
| Rearrange-140                            | [1, 598, 8, 64, 60]  | 0           |
| Attend-141                               | [1, 8, 64, 60]       | 0           |
| Rearrange-142                            | [1, 60, 512]         | 0           |
| Linear-143                               | [1, 60, 384]         | 196, 608    |
| LinearAttention-144                      | [1, 60, 384]         | 0           |
| RMSNorm-145                              | [1, 60, 384]         | 0           |
| Linear-146                               | [1, 60, 384]         | 591, 360    |
| GELU-147                                 | [1, 60, 1536]        | 0           |
| Dropout-148                              | [1, 60, 1536]        | 0           |
| Linear-149                               | [1, 60, 384]         | 590, 208    |
| Dropout-150                              | [1, 60, 384]         | 0           |
| FeedForward-151                          | [1, 60, 384]         | 0           |
| RMSNorm-152                              | [1, 60, 384]         | 0           |
| Transformer-153                          | [1, 60, 384]         | 0           |
| ...                                      |                      |             |
| Linear-314                               | [1, 299, 1536]       | 591, 360    |
| Tanh-315                                 | [1, 299, 1536]       | 0           |
| Linear-316                               | [1, 299, 1536]       | 2, 360, 832 |
| Tanh-317                                 | [1, 299, 1536]       | 0           |
| Linear-318                               | [1, 299, 1536]       | 86, 072     |
| GLU-319                                  | [1, 299, 28]         | 0           |
| ...                                      |                      |             |
| MaskEstimator-674                        | [1, 299, 7916]       | 0           |
| Total params: 228, 144, 088              |                      |             |
| Trainable params: 228, 144, 088          |                      |             |
| Non-trainable params: 0                  |                      |             |
| Input size (MB): 1.00                    |                      |             |
| Forward/backward pass size (MB): 2892.81 |                      |             |
| Params size (MB): 870.30                 |                      |             |
| Estimated Total Size (MB): 3764.12       |                      |             |

Table 1. The structure of Mel-Band RoFormer Model in this project

About the model's parameters, it will be read from the 'config.yaml' file based on an idea of Solovyev's group [23] research. This work will ease the parameters reusable and model modification. Some main parameters can be listed such as:

- **dim, depth, heads**: the capacity and complexity of the model
- **time\_transformer\_depth**: transformer layers in time axis
- **freq\_transformer\_depth**: transformer layers in frequency axis
- **num\_bands, dim\_freqs\_in**: the mel-band processing parameters that effecting frequency decomposition, with 60 and 1025 as the default value for num\_bands and dim\_freqs\_in, respectively
- **attn\_dropout**: Attention layer dropout rate, with 0 as default value
- **ff\_dropout**: Feed-forward layer dropout rate, with 0 as default value
- **dim\_head**: one of the attention mechanism parameters that effects the efficiency and quality of the mechanism. Its default value is set as 64
- **flash\_attn**: Enable flash attention optimization, with True as default value
- **stereo, num\_stems**: the audio processing parameters that effect the model input/output configuration.
- **sample\_rate**: audio sampling rate in Hz, with 44100 was set as a value in most of the model configurations
- **multi\_stft\_resolutions\_window\_sizes**: a tuple of STFT window sizes used for training loss calculation

For the projected model, its parameters will be resued from the parameters of Kimberley Jensen's model [27], which is considered as one of the outstanding experimental result about Mel-Band RoFormer model. The list of model parameters in this project can be list as below:

- dim: 384
- depth: 6
- stereo: True
- num\_stems: 1
- time\_transformer\_depth: 1
- freq\_transformer\_depth: 1
- num\_bands: 60
- dim\_head: 64
- heads: 8
- attn\_dropout: 0
- ff\_dropout: 0
- flash\_attn: True
- dim\_freqs\_in: 1025
- sample\_rate: 44100
- stft\_hop\_length: 441
- stft\_win\_length: 2048
- stft\_normalized: False
- mask\_estimator\_depth: 2
- multi\_stft\_resolution\_loss\_weight: 1.0
- multi stft\_resolutions\_window\_size: 4096 2048 1024 512 256

- multi\_stft\_hop\_size: 147
- multi\_stft\_normalized: False

### 3.2. DATASET

The dataset used in this project is the MUSDB18-HQ [21] dataset, including a total of 150 full-track songs in different styles and includes both the stereo mixtures and the original sources.

MUSDB18-HQ is divided into “train” and “test” folder. The “train” folder is used for training model, including 100 songs, while the “test” folder includes 50 songs uses for testing.

Every file in the MUSDB18-HQ datasets is saved as uncompressed wav files. Each track folder includes five .wav files:

- mixture.wav
- drums.wav
- bass.wav
- other.wav
- vocal.wav

Every signals are stereophonic and encoded at 44.1kHz.

The list of song folders in ‘train’ dataset is shown as Figure 6:

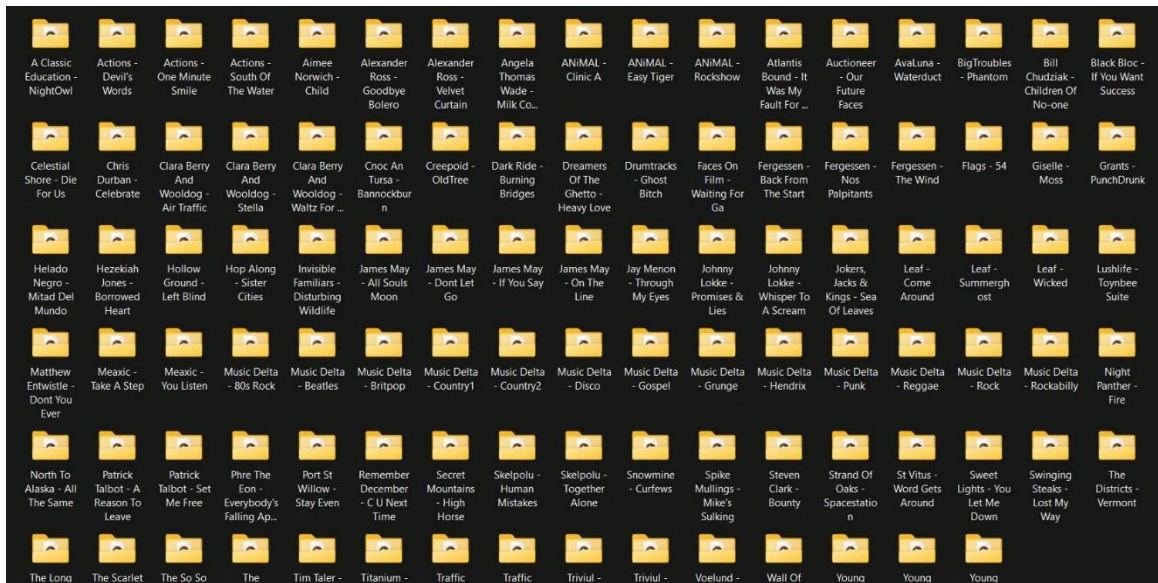


Figure 6. The list of song folders in the MUSDB18-HQ’s train dataset

The list of song folders in ‘test’ dataset is shown as Figure 7:

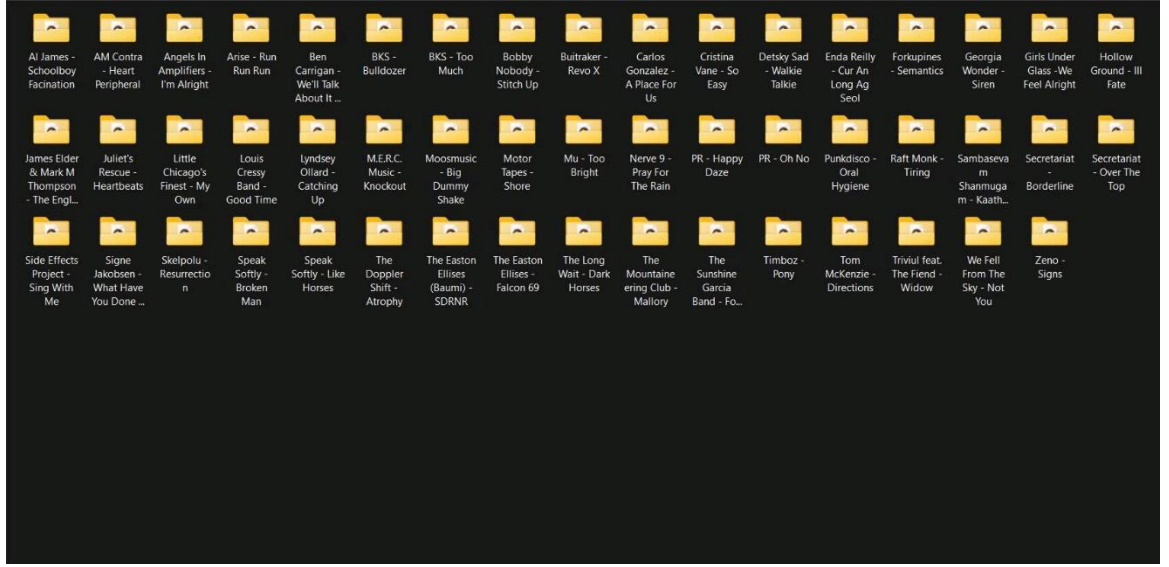


Figure 7. The list of song folders in the MUSDB18-HQ's test dataset

### 3.3. PREPROCESSING DATASET

The audio files in MUSDB18-HQ datasets need to be preprocessed to normalize and simplify data before training.

This process will be performed through dataset.py file. This file will read each sound folders inside the train dataset to extract some important information such as the file path and track length. Then, the necessary file of stems such as vocal and instrumental stems will be processed and mixed into a summed.

The audio signal is then processed through STFT by dividing the signal into smaller frames and applying fast Fourier Transform on each frame to find the frequency components in each moment. The output of the algorithm is a visualized diagram called spectrogram, used for representing the intensity of the audio signal.

### 3.4. TRAINING PROCESS

#### 3.4.1. Device Information

The model will be trained on Kaggle platform, using 2 Tesla T4 GPU with 16GB of VRAM as a runtime option. In the validation and testing, the model will be deployed on the Google Colab with GPU NVIDIA Tesla T4 16GB GDDR6.

#### 3.4.2. Dataset Splitting

The MUSDB18-HQ [21] dataset had been split into two smaller datasets, with 100 sound folders for training and 50 folders for testing. Especially, in the testing dataset, only the music source file named 'mixture.wav' in each song folder will be used to separate the sound data into the vocal-contained file for model validation.

With the purpose of small-scale experimental as well as the limitation of hardware processing performance, only 66 folders will be used for both training and

testing processes. In detail, 50 song folders will be used for training and 16 song folders for testing.

The list of song folders in ‘train’ dataset is shown as Figure 8:

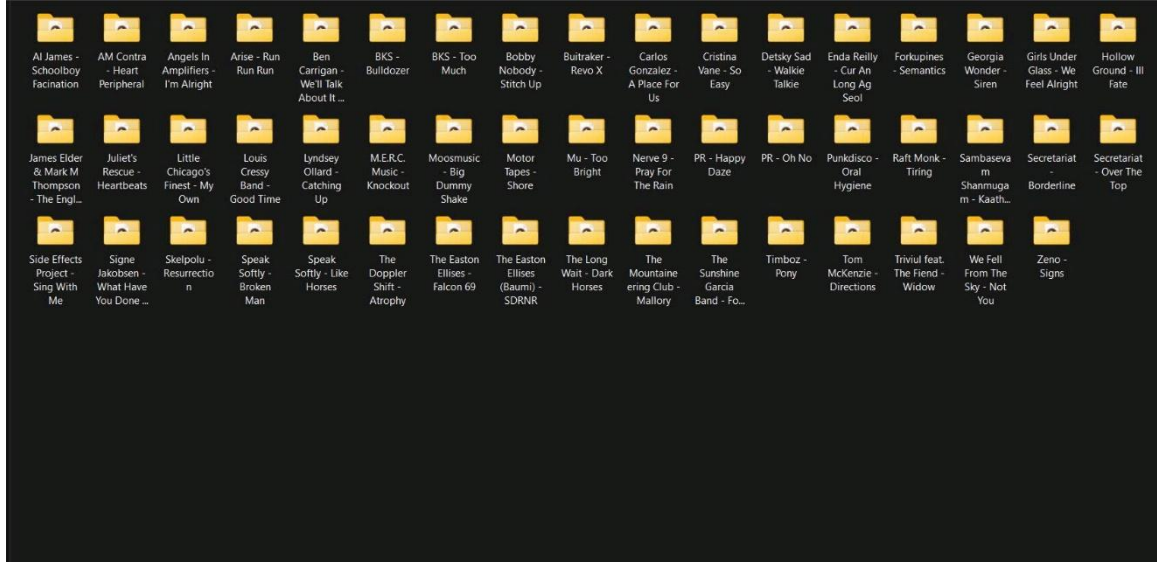


Figure 8. The list of song folders used for training process

The list of song folders in ‘test’ dataset is shown as Figure 9:

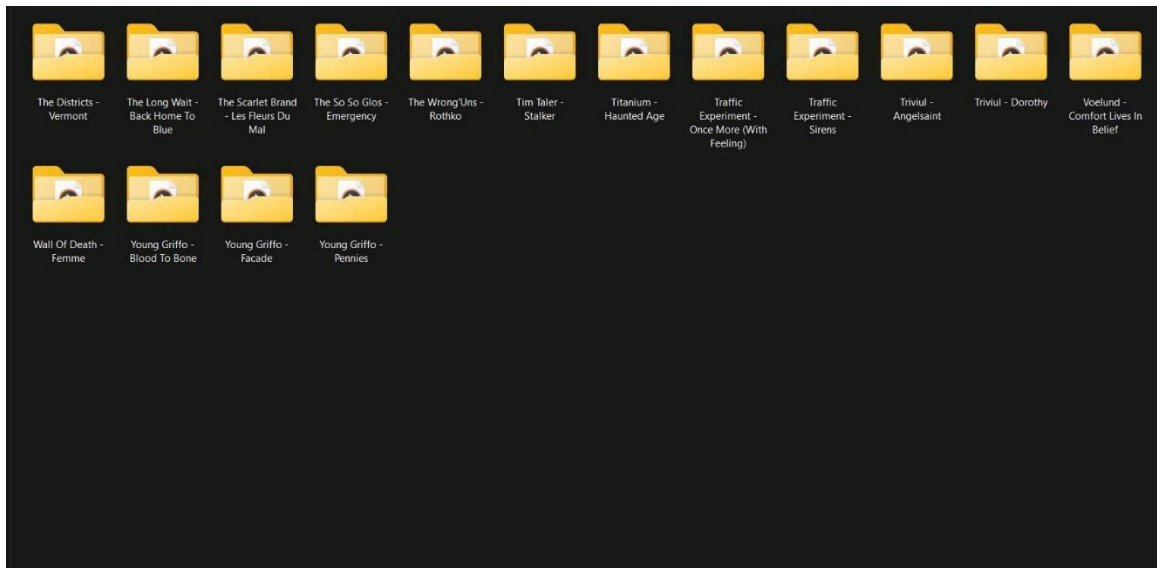


Figure 9. The list of song folders used for testing process

### 3.4.3. Training Parameters

As same as the model parameters, config.yaml file also includes the parameters for training process. This work will also simplify the experiment reproducibility and modification. By providing a wide choice of parameters for different MSS model

families, these parameters for the model is also persistently aggregated from the previous experimented results among the community.

The parameters used for training model includes:

- `batch_size`: 1
- `gradient_accumulation_steps`: 1
- `grad_clip`: 0
- `instruments`: vocals other
- `lr`: 1.0e-04
- `patience`: 2
- `reduce_factor`: 0.95
- `target_instruments`: vocals
- `num_epochs`: 1000
- `num_steps`: 1000
- `q`: 0.95
- `coarse_loss_clip`: false
- `ema_momentum`: 0.999
- `optimizer`: adam
- `other_fix`: false
- `use_amp`: true

Where:

- ***batch\_size***: number of samples per batch.
- ***lr***: initial learning rate
- ***num\_epochs***: number of training epochs
- ***q***: the value of quantile loss threshold, uses to discarded the largest errors in the robust quantile-masked MSE (`masked_loss`)
- ***optimizer***: the name of optimizer used to employ adaptive descent methods. According to [23], Adam/AdamW is recommended as the standard starting optimizer.
- ***patience***, ***reduce\_factor***: two parameters that support for ReduceLROnPlateau scheduler, reducing LR when metric progress stalls.
- ***other\_fix***: a parameter needed for checking on multisong dataset if other is actually instrumental
- ***use\_amp***: a parameter that enable or disable usage of mixed precision (float16), it is usually set as true value.

#### 3.4.4. Deployed Steps

- **Step 1**: Prepare a model with parameters read from the ‘`config.yaml`’ file.
- **Step 2**: If there are any pre-trained model previously, load the model state and its parameters into our model.
- **Step 3**: Prepare the datasets use in training and validating process. For training dataset, read through the root dataset to extract basic information of the songs. Then preprocessing them to combined into the summed signal for the later training phase. In a while, the validation dataset is prepared by

collecting 'mixture.wav' file from the song folders contained in the root dataset for annual separation and validation.

- **Step 4:** Pass necessary optimizer and scheduler into our model to prepare for training.
- **Step 5:** Update the training state from the model to avoid training again from the beginning. For the detail, we will get some information about the optimizer and scheduler state, the current training epoch of the model, the evaluation metrics and all of loss values stored over some previous epoch.
- **Step 6:** We'll start to train our model. In each epoch, each duo (batch, mixes) from the training dataset will be passed into the model to process, learn and calculate the loss value. After that, the training loss will be calculated by the average of every loss value generated from learning these (mixes, batch) duo.
- **Step 7:** After each epoch, the parameters of our model will be updated and added into the checkpoint file (.ckpt).
- **Step 8:** After each multiple epochs period, the model will be annually evaluated to help tracking the changes of the model during the training process. In this step, each collected 'mixture.wav' file will be separated into an audio of vocal stem and evaluated with a list of chosen metrics.

Here is the syntax of the command used for training process:

```
python train.py \  
    --config_path config.yaml \  
    --model_path /path/to/model_checkpoint \  
    --result_path /path/to/result_folder \  
    --data_path path/to/dataset \  
    --valid_path path/to/valid_dataset \  
    --num_workers 4 \  
    --device_ids 0 \  
    --metrics list-of-metrics \  
    --load_epoch \  
    --load_scheduler \  
    --load_optimizer \  
    --load_best_metric \  
    --load_all_losses \  
    --load_all_metrics
```

Where:

--*config\_path*: a path leads to configuration file

--*model\_path*: an initial checkpoint file (.ckpt) to start training

--*valid\_path*: a path leads to validation data

--*result\_path*: a path to folder where the result will be stored, including the model weights and training metadata

--*num\_workers*: the number of workers for loading dataset

--*device\_ids*: a list of GPU Ids

--*metrics*: a list of metrics to use

--*load\_epoch*: load epoch number from the model checkpoint if it is available (Optional)

`--load_scheduler`: load scheduler state from the model checkpoint if it is available (Optional)  
`--load_optimizer`: load scheduler state from the model checkpoint if it is available (Optional)  
`--load_best_metric`: load best metric from the model checkpoint if it is available (Optional)  
`--load_all_losses`: load all losses from the model checkpoint if it is available (Optional)  
`--load_all_metrics`: load all metrics from the model checkpoint if it is available (Optional)

### **3.5. EVALUATE MODEL**

In this step, the SDR metrics from [18] research will be use as the main evaluation metric. After each 5 epochs during training, this metric will be use to validate the separation performance over each song from the test dataset. Besides, some other metrics such as SI-SDR, L1Freq, Fullness and Bleedless will be also applied during the validated process.

## CHAPTER 4. EXPERIMENT AND EVALUATE

### 4.1. MODEL RESULT

In validation phase, the model is evaluated through the ‘test’ folder of the abbreviate version of MUSDB18-HQ dataset. This sub-dataset includes 16 song folders, each folder has 5 sound files with the content of each stems in a song. However, only the ‘mixture.wav’ file of each folder is used for vocal separation and validated process.

Besides, during the training process, the validation work is periodically deployed after each 5 epochs. This work will help minimizing the interrupted time between two epochs and more focus on training model. The testing results of the model is compared and evaluated in different epoch milestones and different evaluated metrics.

Here is the syntax of the command used for validating process:

```
python valid.py \
  --config_path config.yaml \
  --model_path /path/to/model_checkpoint \
  --output_dir /path/to/output_folder \
  --data_path path/to/dataset \
  --valid_path path/to/valid_dataset \
  --num_workers 4 \
  --device_ids 0 \
  --metrics list-of-metrics \
```

Where:

--*config\_path*: a path leads to configuration file  
--*model\_path*: an initial checkpoint file (.ckpt) to start training  
--*valid\_path*: a path leads to validation data  
--*output\_dir*: a path to store results as .wav file  
--*num\_workers*: the number of workers for loading dataset  
--*device\_ids*: a list of GPU Ids  
--*metrics*: a list of metrics to use

Table 2 below describes the training results and testing throughout the epoch milestones.

*Table 2. The table of model results comparison in different epochs.*

| N.o | Epochs | Loss     | SDR    | L1Freq  | Bleedless | Fullness |
|-----|--------|----------|--------|---------|-----------|----------|
| 1   | 5      | 0.709691 | 1.0254 | 20.2738 | 8.2672    | 8.0738   |
| 2   | 10     | 0.586390 | 1.3966 | 21.4398 | 7.8500    | 8.7203   |
| 3   | 15     | 0.558272 | 1.4668 | 21.6984 | 8.4489    | 8.7024   |
| 4   | 20     | 0.539246 | 1.5228 | 22.2333 | 8.4118    | 8.4497   |

|   |    |          |        |         |         |        |
|---|----|----------|--------|---------|---------|--------|
| 5 | 25 | 0.524365 | 1.6334 | 22.7017 | 9.7944  | 8.0496 |
| 6 | 30 | 0.505620 | 1.8422 | 22.9095 | 10.7007 | 8.1213 |
| 7 | 35 | 0.486716 | 1.7215 | 23.0916 | 9.0816  | 8.9809 |
| 8 | 40 | 0.492312 | 1.8136 | 23.0713 | 8.5230  | 9.0836 |
| 9 | 45 | 0.499907 | 1.8285 | 22.9940 | 10.6479 | 9.1699 |

In the same time, the result from the Table 2 above is visualize in these diagram below. This will help easily tracking the change and progress of the model during the experiment.

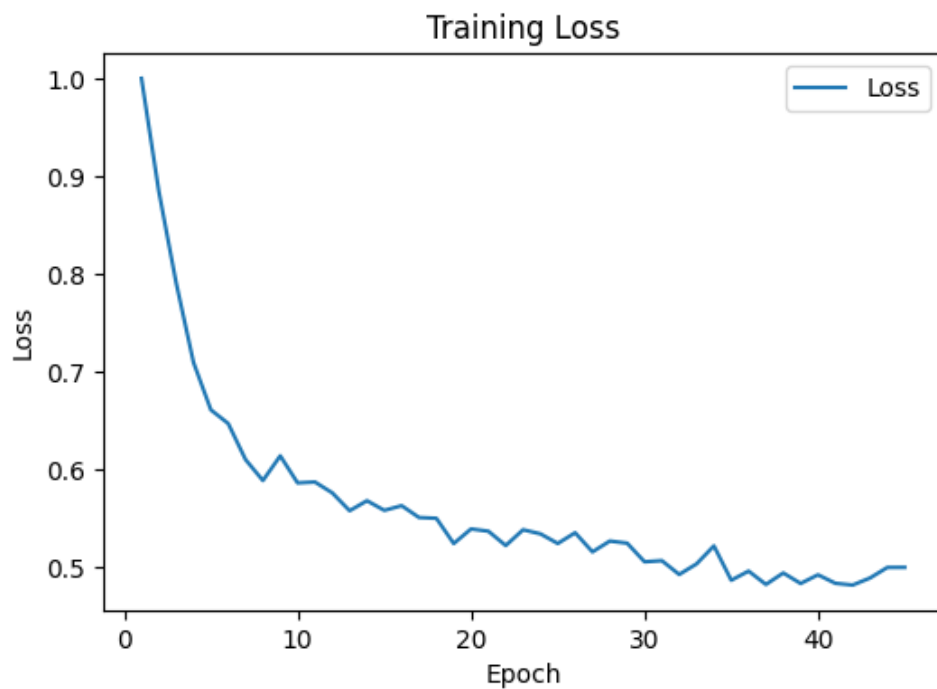
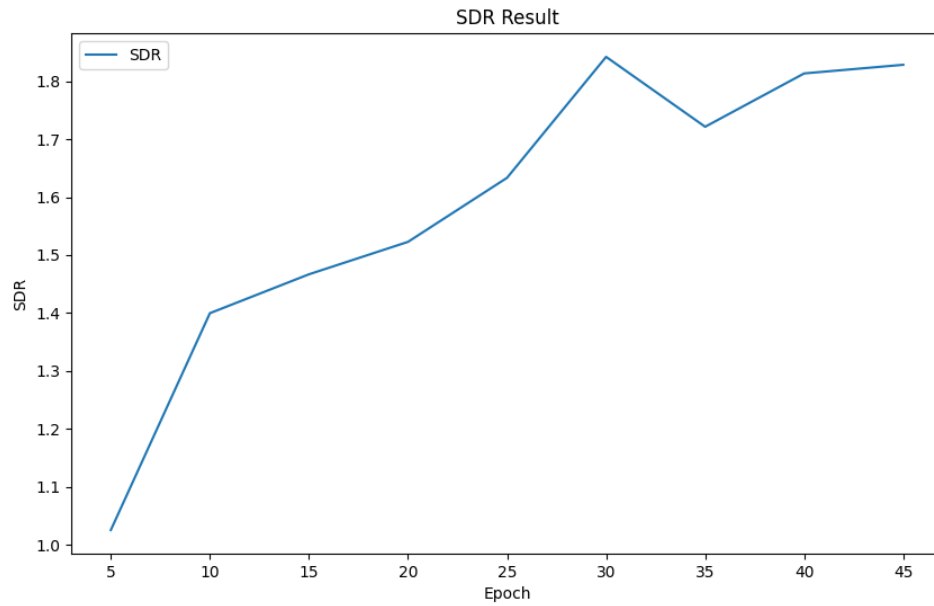


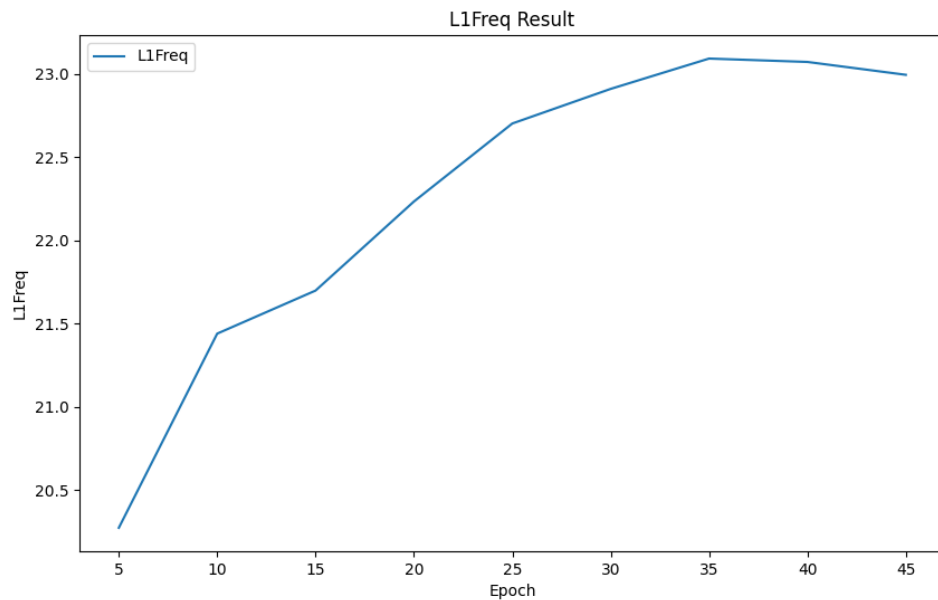
Figure 10. The diagram of training loss until the 45<sup>th</sup> epoch.

As we can see in Figure 10, there was a rapid downfall in the training loss value after 10 first epoch. Until the 35<sup>th</sup> epoch, the loss value had gradually decrease to 0.5 which is near the lowest point of the diagram. From the 36<sup>th</sup> to the 45<sup>th</sup> epoch, this value was seemed fluctuated.



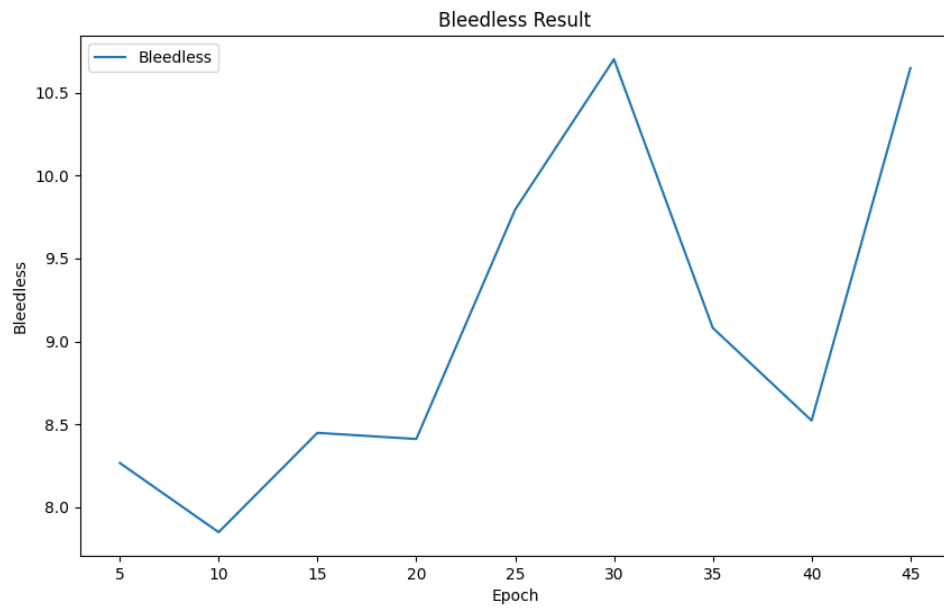
*Figure 11. The diagram of SDR metric values until the 45<sup>th</sup> epoch*

In Figure 11, we can see that there was a rapid rise in the value of SDR metric in the first 10 epochs. From the 11<sup>th</sup> to the 30<sup>th</sup> epoch, this value was steadily risen until reaching the peak of this diagram at over 1.8 in the 30<sup>th</sup> epoch. However, the SDR values was seemed a significant decrease from 1.8 to approximately 1.7 in the 35<sup>th</sup> epoch. Then, in the next 10 epochs, this value had increased again to nearly reach the top of the diagram again in the nearest epoch.



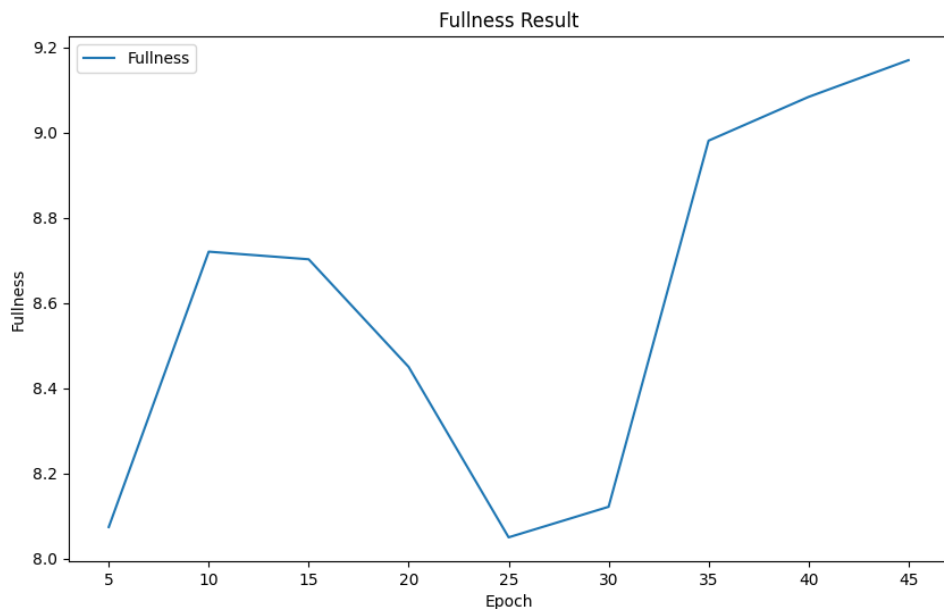
*Figure 12. The diagram of L1Freq metric values until the 45<sup>th</sup> epoch*

It is easily seen that there was a significant rise on the first 35 epochs. The highest value was seen at about 23.0. In the next 10 epochs, this value had lightly fallen.



*Figure 13. The diagram of Bleedless metric values until the 45<sup>th</sup> epoch*

As we can see in Figure 13, there was a complication in the Bleedless metric values trend after 45 epochs. In the first 15 epochs, this value had fluctuated until reaching 8.5 in the 15<sup>th</sup> epoch. It wasn't nearly seen any significant change in the next 5 epochs. However, from the 20<sup>th</sup> to the 30<sup>th</sup> epoch, Bleedless value had a rapid rise to reach the peak of the diagram at 10.5 in the 30<sup>th</sup> epoch, but started to fall again in the next 10-epoch period. In five latest epochs, this value had increased one more time to nearly reach the top of the diagram again.



*Figure 14. The diagram of Fullness metric values until the 45<sup>th</sup> epoch*

Having an opposite value trend with the diagram in Figure 13, the Fullness value in Figure 14 was seem a significant rise at the beginning of the training method. In the next 15 epochs, this value was started to decrease from

approximately 8.8 to just over 8.0 in the 25<sup>th</sup> epoch. But then, in the second half of the current diagram, the Fullness value had started increasing rapidly to reach the highest value in the latest epoch at about 9.2.

However, when the result of this projected model was compared with other models results, it was seemed some disadvantages of the projected model with the result of others. The results comparison between some models is shown at Table 3 below:

*Table 3. The validated results of some Music Source Separation models*

| <b>Model</b>                                 | <b>SDR</b> | <b>L1Freq</b> | <b>Bleedless</b> | <b>Fullness</b> |
|----------------------------------------------|------------|---------------|------------------|-----------------|
| Mel-Band RoFormer<br>(Kimberley Jensen) [27] | 12.1411    | 54.4889       | 42.4617          | 18.4121         |
| MDX-23                                       | 11.3677    | 50.7508       | 28.5344          | 18.4467         |
| BS-RoFormer                                  | 8.0926     | 44.6563       | 31.0456          | 13.9253         |
| Mel-Band RoFormer<br>(Project) (in epoch 45) | 1.8285     | 22.9940       | 10.6479          | 9.1699          |

## 4.2. EVALUATION

To optimise the experience of music listeners and improve the efficiency in the music processing tasks, the model evaluation will be depended on checking the quality of the outputted audio file of the model, especially the file that contains vocals stem.

The evaluation will be based on the personal feeling to hear that if the separated vocal sound are heard clearly or any crackling was heard.

Here is the syntax of the command used for inferential process:

```
python inference.py \
    --config_path config.yaml \
    --model_path /path/to/model_checkpoint \
    --input_folder /path/to/input_folder \
    --output_dir /path/to/output_folder \
```

Where:

--*config\_path*: a path leads to configuration file  
 --*model\_path*: an initial checkpoint file (.ckpt) to start training  
 --*input\_folder*: a folder with songs to process  
 --*output\_dir*: a path to store results as .wav file

Currently, I had checked the ability of vocal separation on two random song files, including:

- **Closer.mp3**: contains the audio sound of the song called Closer by the American music duo The Chainsmokers featuring the female singer named

Halsey. The duration of this file is 4 minutes and 6 seconds. In there, this song has a deep male voice and a mid-range female voice.

- **Chuyện rằng.mp3**: contains the audio of the Vietnamese song called “Chuyện rằng” by Vietnamese singer named Thịnh Suy. This file has a duration of 4 minutes and 32 seconds and includes 1 tenor voice.

Because these audio files are in the .mp3 format, they need to be transformed into the .wav format before processing the separation task.

The vocal audio output of these two songs had shown the low quality. In both two songs, the separated vocal was heard too much crackle and the lyric wasn't heard clearly. In the first song, the deep voice from the male singer hadn't separated well that the separated sound was heard too robotic while the female vocal was heard a bit better with clear melody and lyric in some chunks of the whole song. In the second song, the quality of the separation was seen a bit better when the melody and the vocal lyric was heard clearly. But, these vocals stem was still mixed with some guitar stems at the instrumental sound.

Overall, the ability in separating the vocal stem from the song of the model was still in the low level. The content of these files shown that the separated sounds was heard too much crackle and also mixed with some other stems of the song. Some files can only heard the clear melodies and lyric in some small chunks of the entire song. This disadvantages had led to a solution for longer training phase with a significant number of epochs to improve the quality of the separated sounds at the output.

### 4.3. DISCUSSION

In this part, I will focus on some disadvantages during making the training and evaluated process.

Firstly, the model's current result was seemed too much disadvantages compared to other models. The reason for this result is the model hasn't done the training process in the number of epochs as it was set in 'config.yaml' file (1000 epochs). Besides, setting parameters for the model and the training process through 'config.yaml' file sometimes unsuitable with some accelerator configuration on some open-cloud platform such as Kaggle or Google Colab. For example, the value of batch\_size variable was set as 1 instead of 7 or 4 in the parameters of other models. It was seen that setting up the value of batch\_size variable too big had led to the allocated memory for storing model weights, activations, or intermediate tensors exceed the allowed allocated memory of VRAM (Video RAM).

Second, the ability in separating the vocal stem from the song of the model was still in the low level. This limited point had led to a solution for longer training phase with a significant number of epochs to improve the quality of the separated sounds at the output.

Besides, the validated methods hadn't had a diversity and objectivity yet. In detail, the result of the model was only evaluated on some specific metrics and datasets. And also, model output validation was based only on a subjective personal

view instead of the larger amount of listeners. Therefore, the methods can't show the efficiency of the model in music processing tasks and listeners' experiences.

In overall, there are some experiences can be derived from working on the project. Firstly, it is necessary to change and improve the parameter of the model as well as the training process that fit with the configuration of some accelerator in some open-cloud Notebook environment such as Google Colab or Kaggle. And then, the model needs to be validated on multiple datasets and metrics, accompanies with different survey methods on a large amount of people to evaluate the output vocal audio and increase the objectivity of the result.

## **CHAPTER 5. CONCLUSION**

### **5.1. OVERALL**

After three months of working on projects, the project had gained some achievements.

- Discovered about some well-known deep learning model used for music source separating tasks based on some recent papers.
- Learned about how to apply Transformers architecture to fit with the project topic and process the music source datasets.
- Researched about the method of preprocessing the song audio files before the training process.
- Explored some specific evaluation metrics for validating the separated audio content in the MSS tasks.

However, there were still some problems and limitations that are needed to deal with and solved in the future:

- The model hadn't done the training process in the number of epochs as it was set in 'config.yaml' file. This reason had led to an unwell result as the initial target.
- The amount of data for training was too little due to the limitation of time, belong with the little numbers of evaluation metrics used for validation process. During this project, there wasn't metric used to validate the accuracy of the model during the training process.
- The validation on the model's output was based only on a subjective personal view instead of the larger amount of listeners. This coarse method can't show the efficiency of the model in music processing tasks and listeners' experiences.

### **5.2. DEVELOPMENT DIRECTION**

In the future, I expect to improve the recent results with some potential directions. Firstly, the model is need to be trained in a significant number of epochs to improve the quality of the separated vocal. Secondly, it is great to expand the training dataset to improve the diversity and accuracy of the model. Thirdly, some fine-tuned methods need to be researched and applied for improving the result of the model. One more thing, the result of the model needed to be validated on the larger amount of listeners via some survey. And finally, the model can be move to the higher level by integrating in the web or mobile application.

## REFERENCES

- [1] Âm Nhạc Bắt Nguồn Từ Đâu: Lịch Sử Phát Triển Và Những Thay Đổi, VTH Online, 2023. <https://vth.info.vn/am-nhac-bat-nguon-tu-dau/> [Accessed May 30<sup>th</sup>, 2026]
- [2] The Evolution Of Music Production Software, Serenade, 2024. <https://serenademagazine.com/the-evolution-of-music-production-software/> [Accessed May 30<sup>th</sup>, 2026]
- [3] Pritish Chandna, Marius Miron, Jordi Janer, Emilia Gómez, “Monoaural Audio Source Separation Using Deep Convolutional Neural Networks” in *Latent Variable Analysis and Signal Separation (LVA/ICA)*, 2017, pp. 258-266. [https://link.springer.com/chapter/10.1007/978-3-319-53547-0\\_25](https://link.springer.com/chapter/10.1007/978-3-319-53547-0_25) [Accessed May 26<sup>th</sup>, 2026]
- [4] Andreas Jansson, Eric Humphrey, Nicolas Montecchio, Rachel Bittner, Aparna Kumar, Tillman Weyde, “Singing Voice Separation With Deep U-Net Convolutional Networks” in *International Society for Music Information Retrieval Conference*, 2017, pp. 745-751, <https://archives.ismir.net/ismir2017/paper/000171.pdf> [Accessed May 26<sup>th</sup>, 2026]
- [5] Yi Luo, Jianwei Yu, “Music Source Separation with Band-split RNN” in *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2022, <https://arxiv.org/pdf/2209.15174> [Accessed May 26<sup>th</sup>, 2026]
- [6] Défossez A., Usunnier N., Bouttou L., Bach F., “Music Source Separation In The Waveform Domain”, 2019, <https://arxiv.org/abs/1911.13254>, [Accessed May 18<sup>th</sup>, 2026]
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, “Attention Is All You Need” in *NeurIPS*, vol. 30, 2017, <https://arxiv.org/abs/1706.03762> [Accessed May 18<sup>th</sup>, 2026]
- [8] Wei-Tsung Lu, Ju-Chiang Wang, Minz Won, Keunwoo Choi, Xuchen Song, “SpecTNT: A Time-Frequency Transformer For Music Audio in *ISMIR*, 2021, <https://arxiv.org/pdf/2110.09127> [Accessed May 18<sup>th</sup>, 2026]
- [9] Wei-Tsung Lu, Ju-Chiang Wang, Qiuqiang Kong, Yun-Ning Hung, “Music Source Separation With Band-Split RoPE Transformer” in *ICASSP*, 2023, <https://arxiv.org/abs/2309.02612> [Accessed May 18<sup>th</sup>, 2026]

- [10] Ju-Chiang Wang, Wei-Tsung Lu, Jitong Chen, “Mel-RoFormer For Vocal Separation And Vocal Melody Transcription” in *ISMIR 2024*, 2024, <https://arxiv.org/abs/2409.04702> [Accessed May 18<sup>th</sup>, 2026]
- [11] David Bergmann, “What Is Deep Learning?”, IBM, 2025. <https://www.ibm.com/think/topics/deep-learning> [Accessed May 15<sup>th</sup>, 2026]
- [12] Keiron O’Shea, Ryan Nash, “An Introduction To Convolutional Neural Networks”, 2015, <http://arxiv.org/abs/1511.08458> [Accessed June 16<sup>th</sup>, 2026]
- [13] Robin M. Schmidt, “Recurrent Neural Networks (RNNs): A Gentle Introduction And Overview”, 2019, <https://arxiv.org/pdf/1912.05911> [Accessed June 17<sup>th</sup>, 2026]
- [14] Sepp Hochreiter, Jurgen Schmidhuber, “Long Short-Term Memory” in *Neural computation* 9, 1997, pp. 1735-80, <https://scispace.com/papers/long-short-term-memory-1rd0gpayfc> [Accessed June 17<sup>th</sup>, 2026]
- [15] Short-Time Fourier Transform, Wikipedia, [https://en.wikipedia.org/wiki/Short-time\\_Fourier\\_transform](https://en.wikipedia.org/wiki/Short-time_Fourier_transform) [Accessed June 17<sup>th</sup>, 2026]
- [16] Spectrogram, Wikipedia, <https://en.wikipedia.org/wiki/Spectrogram> [Accessed June 17<sup>th</sup>, 2026]
- [17] Evaluation Metrics in Machine Learning, GeeksForGeeks. <https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/> [Accessed June 17<sup>th</sup>, 2026]
- [18] Emmanuel Vincent, Rémi Gribonval, Cédric Févotte, “Performance Measurement In Blind Audio Source Separation” in *IEEE Transactions On Audio, Speech, And Language Processing*, Vol. 14, No.4, 2006, <https://scispace.com/papers/performance-measurement-in-blind-audio-source-separation-4la127li8d> [Accessed May 26<sup>th</sup>, 2026]
- [19] Google Colab. <https://research.google.com/colaboratory/faq.html> [Accessed June 17<sup>th</sup>, 2026]
- [20] Kaggle. <https://en.wikipedia.org/wiki/Kaggle> [Accessed June 17<sup>th</sup>, 2026]
- [21] Zafar Rafil, Antoine Liutkus, Fabian-Robert Stoter, Stylianos I. Mimilakis, Rachel Bittner, “The MUSDB18 corpus for music separation”, 2017, <https://zenodo.org/records/3338373> [Accessed June 16<sup>th</sup>, 2026]

- [22] Giorgio Fabbro, Stefan Uhlich, Chieh-Hsin Lai, Woo-sung Choi, Marco Martínez, Weihsiang Liao, Igor Gadelha, Geraldo Ramos, Eddie Hsu, Hugo Rodrigues, Fabian-Robert Stöter, Alexandre Défossez, Yi Luo, Jianwei Yu, Dipam Chakraborty, Sharada Mohanty, Roman Solovyev, Alexander Stempkovskiy, Tatiana Habruseva, Nabarun Goswami, Tatsuya Harada, Min-seok Kim, Jun Hyung Lee, Yuanliang Dong, Xinran Zhang, Jiafeng Liu, Yuki Mitsufuji, “The Sound Demixing Challenge 2023 – Music Demixing Track” in *Transactions of the International Society for Music Information Retrieval*, 2023, <https://arxiv.org/pdf/2308.06979> [Accessed May 28<sup>th</sup>, 2026]
- [23] Roman Solovyev, Ilya Kiselev, Alexander Stempkovskiy, Tatiana Gabruseva, “Music-Source-Separation-Training (MSST): A Unified Framework For Training And Evaluating Music Demixing Models”, 2026, <https://arxiv.org/pdf/2607.23395>
- [24] Roman Solovyev and Ilya Kiselev, “Music-source-separation-training: Universal training code for music source separation”, GitHub Repository, 2023, <https://github.com/ZFTurbo/Music-Source-Separation-Training> [Accessed July 31<sup>st</sup>, 2026]
- [25] Jonathan Le Roux, Scott Wisdom, Hakan Erdogan, John R. Hershey, “SDR – Half-Baked Or Well Done?”, 2018, <https://arxiv.org/pdf/1811.02508> [Accessed July 31<sup>st</sup> 2026]
- [26] Enric Gusó, Jordi Pons, Santiago Pascual, Joan Serra, “On Loss Functions And Evaluation Metrics For Music Source Separation” in *IEEE International Conference on Acoustics, Speech, and Signal Processing*, 2022, <https://arxiv.org/pdf/2202.07968> [Accessed August 1<sup>st</sup>, 2026]
- [27] Kimberley Jensen, “Mel-Band-Roformer Vocal Model, GitHub Repository, 2023, <https://github.com/KimberleyJensen/Mel-Band-Roformer-Vocal-Model> [Accessed August 1<sup>st</sup>, 2026]